

SPRINT 7

Resoldràs alguns problemes de la vida quotidiana aplicant les estructures de dades i control en Python.

Un client de l'empresa en la qual treballes demana una llista de programes molt senzills, però que li facilitarien molts processos. No obstant això, el departament de TU està molt complicat amb el temps, per la qual cosa et demanen que facis la programació.

Nivell 1

Exercici 1

Calculadora de l'índex de massa corporal:

- Escriu una **funció** que calculi l'IMC ingressat per l'usuari/ària, és a dir, qui ho executi haurà d'ingressar aquestes dades.
- La funció ha de classificar el resultat en les seves respectives categories.

```
def calcular_IMC(peso, altura):  
    IMC = peso / (altura ** 2)  
  
    if IMC <= 18.5:  
        clasificacion = "BAJOPESO"  
    elif 18.5 < IMC <= 24.9:  
        clasificacion = "NORMOPESO"  
    elif 25 <= IMC <= 29.9:  
        clasificacion = "SOBREPESO"  
    else:  
        clasificacion = "OBESIDAD"  
  
    return IMC, clasificacion  
  
peso = float(input("Introduzca su peso en kg: "))  
altura = float(input("Introduzca su altura en metros: "))  
  
IMC, clasificacion = calcular_IMC(peso, altura)  
print(f"Tu IMC es: {IMC:.2f} y tu clasificación es {clasificacion}")  
  
[9]  
... Tu IMC es: 23.03 y tu clasificación es NORMOPESO
```

En el primer ejercicio creé una función llamada "calcular_IMC" que, además de hacer el calculo del IMC (una vez dados a través del input los datos del peso en kg y la altura en metros de la persona), clasifica el resultado obtenido en bajopeso, normopeso, sobrepeso u obesidad según los parámetros marcados por la OMS.

Para poder trabajar con los datos proporcionados a través del input, y dado que al introducir datos de esa manera python los considera texto (str) y no números, convertí ambos inputs en números decimales (float) ya que algunos de los datos pueden contener comas.

Por último, se imprime un mensaje con el resultado del cálculo del IMC y la clasificación. En el ejemplo yo introduje de peso: 65kg y de altura 1.68m, y acoté el resultado del cálculo del IMC a 2 decimales utilizando {IMC:.2f}.

Exercici 2

Convertidor de temperatures:

- Existeixen diverses unitats de temperatura utilitzades en diferents contextos i regions. Les més comunes són Celsius (°C), Fahrenheit (°F) i Kelvin (K). També existeixen altres unitats com Rankine (°Ra) i Réaumur (°Re). Selecciona almenys 2 conversors, de tal manera que en introduir una temperatura retorni, com a mínim, dues conversions.



```
def convertidor_temperatura(celsius):
    fahrenheit = (celsius * 9/5) + 32
    kelvin = celsius + 273.15
    rankine = (celsius + 273.15) * 9/5
    reaumur = celsius * 0.8

    return fahrenheit, kelvin, rankine, reaumur

celsius = float(input("Introduzca la temperatura en grados celsius: "))

fahrenheit, kelvin, rankine, reaumur = convertidor_temperatura(celsius)

print(f"{celsius}°C equivalen a {fahrenheit:.2f}°F, {kelvin:.2f}K, {rankine:.2f}°Ra y {reaumur:.2f}°Re")
```

[5] ✓ 3.2s

... 18.0°C equivalen a 64.40°F, 291.15K, 524.07°Ra y 14.40°Re

En este segundo ejercicio creé una función llamada “convertidor_temperatura” que, a través de cálculos matemáticos, convierte una temperatura en °C (dada a través de un input) en las otras 4 unidades de temperatura sugeridas en el enunciado (°F, K, °Ra, °Re).

Al igual que en el ejercicio anterior he de convertir el input en un float para poder realizar calculos matemáticos con él y al final de todo se imprimirá un mensaje con la temperatura introducida en grados celsius seguida de la conversión en las otras cuatro unidades de temperatura. En este ejemplo yo introduje 18°C y también acoté el resultado de las diferentes temperaturas a 2 decimales utilizando por ejemplo {fahrenheit:.2f} en el caso de los grados fahrenheit.

Exercici 3

Comptador de paraules d'un text:

- Escriu una funció que donat un text, mostri les vegades que apareix cada paraula.

En el tercer ejercicio, antes de crear mi función, importé “re” una librería que nos permite trabajar con expresiones regulares: buscando, reemplazando, dividiendo y manipulando cadenas de texto de manera

avanzada, para luego poder utilizar `re.sub()` que nos permite eliminar todos los signos de puntuación de nuestro texto.

```
import re # importo el módulo re primero para poder utilizar la función re.sub().
# re es la biblioteca estándar para trabajar con expresiones regulares (regex): permite buscar, reemplazar, dividir y manipular cadenas de texto de manera avanzada.

def contador_palabras(texto):
    texto = texto.lower() # convertimos el texto a minúsculas
    texto = re.sub(r'[^\w\s]', '', texto) # eliminamos los signos de puntuación pero no los espacios

    palabras = {}
    lista_palabras = texto.split() # dividimos el texto en palabras

    for palabra in lista_palabras:
        if palabra in palabras:
            palabras[palabra] += 1
        else:
            palabras[palabra] = 1

    return palabras

texto = input("Introduzca un texto: ")

palabras = contador_palabras(texto)

print(palabras)
```

[20]

```
... {'que': 1, 'toda': 1, 'la': 1, 'vida': 1, 'es': 1, 'sueño': 1, 'y': 1, 'los': 1, 'sueños': 2, 'son': 1}
```

Luego creé la función “contador_palabras”, la cuál dado un texto (utilizando el input) primero lo convierte todo en minúsculas con `.lower`, luego elimina los signos de puntuación (con `re.sub`, como ya había comentado antes) y luego con `.split()` lo dividimos en las diferentes palabras que lo componen, creando también un diccionario vacío llamado “palabras”. Por último, utilizando un condicional dentro de un bucle for rellenamos el diccionario vacío haciendo un conteo de las repeticiones de cada palabra de nuestro texto: si la palabra no estaba antes en el diccionario se añade como key con un value de 1, y si la palabra ya había sido añadida antes se aumenta en 1 el value.

Utilizando un input se pide que se introduzca un texto y se imprime el diccionario del conteo de palabras al final. En el ejemplo yo utilicé la frase: “Que toda la vida es sueño y los sueños, sueños son...”.

Exercici 4

Diccionari invers:

- Resulta que el client té una enquesta molt antiga que s'emmagatzema en un diccionari i els resultats els necessita al revés, és a dir, intercanviats les claus i els valors. Els valors i claus en el diccionari original són únics; si aquest no és el cas, la funció hauria d'imprimir un missatge d'avertiment:

```
reverse_dictionary({'a': 1, 'b': 2, 'c': 3})

{1: 'a', 2: 'b', 3: 'c'}
```

```
reverse_dictionary({'x': 'apple', 'y': 'banana', 'z': 'banana'})

Error: multiple keys for one value
```

En este ejercicio, creé una función llamada “diccionario_inverso” en la cual añadí un “diccionario_nuevo” vacío para almacenar nuestro diccionario inverso y luego utilicé un condicional dentro de un bucle for que itera para cada key y value de nuestro diccionario (utilizando la función .items()) para forzar la obtenención de un mensaje de error. Normalmente si un diccionario tiene dos keys iguales elimina una sin darnos un error por eso se tiene que forzar el mismo y para ello yo utilicé ‘raise’ (dentro del condicional) que se utiliza para lanzar una excepción intencionalmente, seguido de ‘ValueError’ y el mensaje de error que debía salir si el diccionario_nuevo tenía valores repetidos. Por último, si el diccionario_nuevo no tenía valores repetidos simplemente se cambiaba el orden de key y value para obtener nuestro diccionario inverso.

```
def diccionario_inverso(diccionario):
    diccionario_nuevo = {}

    for key, value in diccionario.items():
        if value in diccionario_nuevo:
            raise ValueError(f"Error: múltiples keys para el value '{value}'") # lanza un error
        diccionario_nuevo[value] = key # cambia el orden clave:valor

    return diccionario_nuevo

diccionario = {'x': 'apple', 'y': 'banana', 'z': 'banana'}

# probamos que imprime error en valores duplicados:
try: # usamos try-except para que nos muestre el error si hay valores duplicados
    diccionario_nuevo = diccionario_inverso(diccionario)
    print(diccionario_nuevo)
except ValueError as e:
    print(e)
```

[1] ✓ 0.0s

... Error: múltiples keys para el value 'banana'

Primero introduje un diccionario con un value repetido (banana) y en este caso se puede observar que me sale el error notificando que está repetido.

```
diccionario2 = {'x': 'apple', 'y': 'banana', 'z': 'strawberry'}

# probamos que funciona si no hay valores duplicados:
try:
    diccionario_nuevo = diccionario_inverso(diccionario2)
    print(diccionario_nuevo)
except ValueError as e:
    print(e)
```

[2] ✓ 0.0s

... {'apple': 'x', 'banana': 'y', 'strawberry': 'z'}

Luego lo probé sustituyendo el value repetido y entonces si obtuve mi diccionario invertido.

En este ejercicio para obtener el diccionario_nuevo al final en vez de utilizar print() solamente como en los anteriores, utilicé try-except para que mostrara el error si había valores duplicados.

Nivell 2

Exercici 1

Diccionari invers amb duplicats:

- Continuant amb l'exercici 4 del nivell 1: al client es va oblidar de comentar un detall i resulta que els valors en el diccionari original poden duplicar-se i més, per la qual cosa les claus intercanviades poden tenir duplicats. En aquest cas, en l'exercici anterior imprimies un missatge d'avertiment, ara, els valors del diccionari resultant hauran d'emmagatzemar-se com una llista. Tingues en compte que si és un valor únic no ha de ser una llista.

```
reverse_dictionary_plus({'x': 'apple', 'y': 'banana', 'z': 'banana'})

{'apple': 'x', 'banana': ['y', 'z']}
```

Este ejercicio es una modificación del anterior pero en este caso lo que queremos es que en lugar de darnos un error cuando se encuentre con un value duplicado, al hacer la inversión le asigne a dicho value (que ahora es una key) las key que antes lo tenían como value pero en forma de lista dentro del diccionario. Pero todo esto será si hay duplicados, si el valor es único se almacena dentro del diccionario en forma de key:value normal.

```
def diccionario_inverso2(diccionario):
    diccionario_nuevo2 = {}

    for key, value in diccionario.items():
        if value not in diccionario_nuevo2:
            diccionario_nuevo2[value] = key # si es el primer valor, lo guardamos como string
        else:
            if isinstance(diccionario_nuevo2[value], list): # la función isinstance(objeto, tipo) verifica si un objeto es de un determinado tipo
                diccionario_nuevo2[value].append(key) # si ya es una lista, agregamos
            else:
                diccionario_nuevo2[value] = [diccionario_nuevo2[value], key] # si no, convertimos en lista

    return diccionario_nuevo2

diccionario = {'x': 'apple', 'y': 'banana', 'z': 'banana'}

diccionario_nuevo2 = diccionario_inverso2(diccionario)

print(diccionario_nuevo2)
```

[3] ✓ 0.0s

```
... {'apple': 'x', 'banana': ['y', 'z']}
```

Para conseguir estos cambios utilicé el código del ejercicio anterior con algunas modificaciones:

- En la primera condicional del bucle for ahora en lugar de imprimirse el error lo que pasa es que lee si el valor ha sido introducido ya en el diccionario_nuevo2 (que está vacío en un principio) y si no lo ha estado simplemente introduce la key:value cambiando el orden inicial.

- Posteriormente agregué un segundo condicional dentro del primero para que si el valor ya ha sido introducido en el diccionario_nuevo2:
 1. Si ya existe una lista de valores para esa key, añade la ahora value a la lista de valores que comparten la misma key. Para esto utilicé la función `isinstance(objeto, tipo)` que se encarga de verificar si un objeto es de un determinado tipo, en este caso si es una lista.
 2. Si solo hay un value anterior para esa key cree una lista de valores.

(Estas 2 posibilidades se dan a la vez que se cambia el orden key:value)

Al final imprimí el diccionario_nuevo2 y como se puede observar, ya que introduje un value repetido para varias keys en el diccionario original, en el diccionario_nuevo2 se creó una lista para almacenar las keys de dicho value al cambiar el orden key:value.

Exercici 2

Conversió de tipus de dades:

- El client rep una llista de dades i necessita generar dues llistes, la primera on estaran tots els elements que es van poder convertir en flotants i l'altra on estan els elements que no es van poder convertir. Exemple de la llista que rep el client: ['1.3', 'one' , '1e10' , 'seven', '3-1/2', ('2',1,1.4,'not-a-number'), [1,2,'3','3.4']]

```
conversion([ '1.3', 'one' , '1e10' , 'seven', '3-1/2', ('2',1,1.4,'not-a-number'), [1,2,'3','3.4']])

([1.3, 10000000000.0, 2.0, 1.0, 1.4, 1.0, 2.0, 3.0, 3.4],
 ['one', 'seven', '3-1/2', 'not-a-number'])
```

Para poder llevar a cabo este ejercicio tuve que crear dos funciones, la primera para aplanar la lista (ya que dentro de la lista inicial había otra lista y una tupla) y la segunda ya para separar los datos en las dos listas finales.

```
# Primero añado una función para aplanar la lista

def aplanar(lista):
    lista_plana = []
    for elemento in lista:
        if isinstance(elemento, (list, tuple)): # la función isinstance(objeto, tipo) verifica si un objeto es de un determinado tipo
            lista_plana.extend(aplanar(elemento)) # si es una lista o una tupla, la procesamos recursivamente
        else:
            lista_plana.append(elemento) # si no, lo agregamos tal cual
    return lista_plana

lista = ['1.3', 'one', '1e10', 'seven', '3-1/2', ('2', 1, 1.4, 'not-a-number'), [1, 2, '3', '3.4']]

lista_plana = aplanar(lista)
print(lista_plana)
```

[6] ✓ 0.0s

... ['1.3', 'one', '1e10', 'seven', '3-1/2', '2', 1, 1.4, 'not-a-number', 1, 2, '3', '3.4']

Para aplanar la lista cree la función “aplanar” en la cual, dentro de un bucle for que recorre todos los elementos de la lista, introduje un condicional que determina (utilizando la función `isinstance()`) si el elemento se trata de una tupla o una lista. De ser cierto que es una tupla o una lista lo procesamos recursivamente añadiéndolo posteriormente a una lista_plana vacía que cree antes (utilizando la función `.append()`); y si no se trata de una lista o tupla lo añade directamente a la lista_plana vacía (utilizando la función `.append()`). Por ultimo imprimo la lista_plana para confirmar que esté todo correcto.

```

# Después la función para separar los datos de la nueva lista en flotantes o no

def organizador_listas(lista_plana):
    lista_flotantes = []
    lista_otros = []

    for elemento in lista_plana:
        try: # usamos try-except para evitar que el programa falle con valores no numéricos
            num = float(elemento) # intentamos convertir a float
            lista_flotantes.append(num) # si funciona: es un número flotante
        except (ValueError, TypeError): # valor incorrecto o tipo de dato incorrecto para la operación
            lista_otros.append(elemento) # si falla: lo agregamos a la otra lista

    return lista_flotantes, lista_otros

lista_flotantes, lista_otros = organizador_listas(lista_plana)

print(f"lista_flotantes: {lista_flotantes}")
print(f"lista_otros: {lista_otros}")

```

[7] ✓ 0.0s

```

... lista_flotantes: [1.3, 10000000000.0, 2.0, 1.0, 1.4, 1.0, 2.0, 3.0, 3.4]
    lista_otros: ['one', 'seven', '3-1/2', 'not-a-number']

```

Posteriormente, para separar la lista_plana en dos listas finales que almacenaran flotantes y otros creé la función “organizador listas” en la cual primero creé dos listas vacías: lista_flotantes y lista_otros. Después, dentro de un bucle for que recore la lista_plana añadí un try_except para evitar que el programa fallara con valores no numéricos y luego intenté convertir cada elemento en un float (porque los elementos de esta lista_plana son muchos string porque están entre comillas). Si el elemento se convertía en float se añadía, usando la función `.append()`, a la lista_flotantes; si el intento de conversión a float daba error se añadía el elemento, usando la función `.append()`, a la lista_otros.

Por ultimo imprimí ambas listas con su nombre cada una para comprobar que el resultado era correcto.

Nivell 3

Exercici 1

Comptador i endreçador de paraules d'un text:

- El client va quedar content amb el comptador de paraules, però ara vol llegir arxius TXT i que calculi la freqüència de cada paraula ordenades dins de les entrades habituals del diccionari segons la lletra amb la qual comencen, és a dir, les claus han d'anar de la A a la Z i dins de la A hem d'anar de la A la Z. Per exemple, per a l'arxiu "tu_me_quieres_blanca.txt" la sortida esperada seria:

```
{ 'a': { 'a': 3,
      'agua': 1,
      'al': 2,
      'alba': 4,
      'alcobas': 1,
      'alimenta': 1,
      'alma': 1,
      'amarga': 1,
      'azucena': 1},
  'b': { 'baco': 1,
      'banquete': 1,
      'bebe': 1,
      'blanca': 3,
      'boca': 1,
      'bosques': 1,
      'buen': 1},
  'c': { 'cabañas': 1,
      'carnes': 2,
      'casta': 3,
      'cerrada': 1,
      'con': 4,
      'conservas': 1,
      'copas': 1,
      'corola': 1,
      'corriste': 1,
  ...
  'tornadas': 1,
  'tú': 8},
  'u': { 'un': 1, 'una': 1},
  'v': { 'vestido': 1, 'vete': 1, 'vive': 1},
  'y': { 'y': 5}}
```

Para resolver este ejercicio creé una nueva función llamada “contador_texto_ordenado”, pero antes de poder utilizarla tuve que importar de nuevo la librería re (aunque ya lo había hecho antes como mi código está en formato jupyter notebook y hay celdas diferenciadas para cada ejercicio si se quiere ejecutar solo está celda sin ejecutar antes la otra celda donde importé anteriormente esta librería daría problemas, así que por eso la vuelvo a importar). También fue necesario, antes de comenzar a crear mi función, abrir y leer el documento .txt en el cual se encontraba almacenado el mismo para eso utilizamos with open() para abrir el documento y .read() posteriormente para leerlo.

Además, una vez leído el texto pasé a su procesamiento con las funciones .lower(), re.sub() y .replace() convirtiéndolo en minúsculas, eliminando los signos de puntuación y los saltos de línea respectivamente.

Una vez abierto, leído y procesado el texto pasamos a crear nuestra función “contador_texto_ordenado”:

- Lo primero es crear un diccionario vacío “dicc_texto” en el que almacenaremos el resultado y ordenar nuestro texto alfabeticamente utilizando la función sorted() una vez lo hemos dividido en palabras y almacenado en una lista (lista_texto) utilizando .split().

```
import re # importo el módulo re primero para poder utilizar la funcion re.sub()

with open("/Users/liss/Desktop/ItAcademy/2-Especialización/iSprints!/S7/Datos/tu_me_quieres_blanca.txt", "r") as texto2:
    texto2 = texto2.read() # leer el archivo de texto

texto2 = texto2.lower() # Convertimos a minúsculas
texto2 = re.sub(r'[\W\s]', '', texto2) # Eliminamos signos de puntuación
texto2 = texto2.replace("\n", " ") # Eliminamos saltos de línea

def contador_texto_ordenado(texto):
    dicc_texto = {}
    lista_texto = sorted(texto.split()) # dividimos en palabras y ordenamos

    for palabra in lista_texto:
        letra_inicial = palabra[0] # obtener la primera letra

        if letra_inicial not in dicc_texto:
            dicc_texto[letra_inicial] = {} # crear la key si no existe

        if palabra in dicc_texto[letra_inicial]:
            dicc_texto[letra_inicial][palabra] += 1 # si la palabra ya está en el diccionario: aumentar el contador
        else:
            dicc_texto[letra_inicial][palabra] = 1 # si la palabra no está en el diccionario: inicializar el contador

    return dicc_texto

dicc_texto_ordenado = contador_texto_ordenado(texto2)

dicc_texto_ordenado
```

[4] ✓ 0.2s

```
... {'a': {'a': 3,
      'agua': 1,
      'al': 2,
      'alba': 4,
      'alcobas': 1,
      'alimenta': 1,
      'alma': 1,
      'amarga': 1,
      'azucena': 1},
     'b': {'baco': 1,
```

- Después utilizando un bucle for recorreremos la lista_texto y vamos obteniendo la primera letra de cada una de las palabras que la componen, y utilizando un condicional:
 - Si la letra inicial no ha sido añadida anteriormente a dicc_texto la añadimos creando un diccionario vacío a su vez dentro de este con la letra inicial como key del mismo.
 - Si la letra inicial ya está en el diccionario, utilizamos otro condicional dentro de este:

- ❖ Si la palabra que estamos analizando ya está incluida dentro del diccionario que se ha creado con su letra inicial añadimos un contador a su value (asi contamos las apariciones de cada palabra en el texto).
- ❖ Si por el contrario la palabra no está incluida en el diccionario dentro del diccionario de su letra inicial la añadimos como key con value de 1.

Por ultimo imprimimos el `diccionario_texto_ordenado` que contendrá todas las palabras del texto ordenadas dentro de otro diccionario que se encuentra dentro del anterior cuyo nombre se corresponde con la letra inicial de las palabras (ordenadas alfabeticamente como en un diccionario de la RAE) y dentro de cada uno de estos diccionarios dentro del `diccionario_texto_ordenado` tendremos las palabras con su conteo de aparicion en formato `key:value` como se puede observar en el fragmento del resultado de la imagen.